Secret Sauce

Repo-grounded one-page app summary

Generated from repo evidence only

Single-page PDF deliverable

WHAT IT IS

Secret Sauce is a recipe management and smart grocery shopping app that helps users discover recipes, plan meals, track pantry items, and compare grocery prices across stores.

Its core differentiator in the repo is an AI-driven ingredient standardization pipeline that normalizes ingredients across recipes and maps them to real store products.

WHO IT IS FOR

Primary persona: budget-conscious students - especially Berkeley students - who want to eat better, save money, and reduce meal-planning stress.

WHAT IT DOES

- Recipe browsing plus recipe CRUD surfaces (`/recipes`, `/recipes/[id]`, `/upload-recipe`, `/edit-recipe/[id]`).
- Weekly meal planning with premium gating on `/meal-planner`.
- Pantry tracking with pantry-context ingredient standardization via `POST /api/ingredients/standardize`.
- Shopping list management and multi-store price comparison across routes like `/shopping`, `/store`, and `/api/shopping/comparison`.
- Recipe import from URL, Instagram, image/OCR, and paragraph text through `app/api/recipe-import/*` and the Python API.
- User onboarding, settings, tutorials, and account flows using Clerk-backed auth plus profile bridging.
- Premium billing and subscription sync through Stripe checkout and webhook routes.

HOW IT WORKS

- Frontend: Next.js App Router routes in `app/`, shared UI in `components/`, and global providers in `app/layout.tsx` for Clerk, theme, React Query, auth, analytics, and tutorials.
- Client data layer: hooks and `lib/database/*` wrappers read and write app data instead of pages querying tables directly.
- API layer: `app/api/*` handles auth/profile bridging, grocery search and comparison, maps and location, checkout, and recipe-import proxy routes.
- Primary data store: Supabase-backed profile, recipe, shopping, embedding, and canonical ingredient tables referenced throughout `lib/database/*` and docs.
- Import service: `python-api/main.py` exposes FastAPI endpoints for URL scraping, Instagram parsing, OCR/text parsing, and grocery scraper orchestration.
- Background processing: `backend/orchestrators/*` and `backend/workers/*` run ingredient matching, embeddings, vector double-checks, canonical consolidation/medoid, daily scraping, and store maintenance.
- High-level flow: UI -> Next routes or DB wrappers -> Supabase. Import flows -> Next API -> Python API -> external parsers/AI. Workers read and write Supabase queues, embeddings, and canonical tables.

HOW TO RUN

- Install JS dependencies: exact install command is Not found in repo.
- Create `.env.local`: template/example file is Not found in repo. `docs/api-and-integrations.md` lists required Clerk, Supabase, Stripe, Google Maps, and `PYTHON_SERVICE_URL` values.
- Start the web app with `npm run dev`.
- Optional local services: `python-api/main.py` contains the FastAPI import service, and `docker-compose.local.yml` plus `docker/compose/local/*` define worker and scraper stacks.

Explicit Gaps

Install command: Not found in repo

.env example/template: Not found in repo

Python local launch docs: Not found in repo

Sources used: docs/product-overview.md, docs/architecture-and-surfaces.md, docs/api-and-integrations.md, docs/operations-and-workflows.md, package.json, app/layout.tsx, app/home/page.tsx, components/landing/landing-page.tsx, python-api/main.py.